

Universidad de Monterrey



Integración de aplicaciones computacionales

Proyecto Final - Segundo Avance

Reporte Escrito: Predicthealth

Maestro: Dr. Raúl Morales Salcedo

Equipo #3

Margarita Concepción Cuervo Citalán #581771

Mariana Samperio Cuevas # 639835

Bryan Andrés Ramírez Palacios #685060

Día y hora del grupo de la clase: lunes y jueves 10:00h

11 de octubre de 2025.

Damos nuestra palabra de haber realizado esta actividad con integridad académica.

Índice

1. Introducción.....	2
2. Objetivo del Back End CMS.....	3
3. Funcionalidades implementadas.....	4
4. Queries SQL y Stored Procedures.....	8
5. KPIs, métricas y gráficas.....	17
6. Prototipo en funcionamiento.....	22
7. Conclusiones.....	28

Introducción

Este reporte presenta la implementación del módulo Back End del sistema de gestión de contenidos (CMS) correspondiente al proyecto *Predict Health*, una plataforma orientada a la administración y análisis de información clínica, con el objetivo de facilitar la predicción de diagnósticos médicos. El sistema desarrollado proporciona una base robusta para la gestión eficiente de datos vinculados a pacientes, profesionales de la salud, instituciones y resultados de predicción, permitiendo al personal autorizado interactuar con la información de manera segura, estructurada y analítica.

En este contexto, el documento describe las funcionalidades implementadas, el diseño y uso de componentes SQL avanzados, la lógica de negocio aplicada, así como la interpretación de las gráficas generadas. También se incluyen capturas de pantalla del sistema en funcionamiento y una reflexión final sobre el proceso de desarrollo y los principales desafíos enfrentados durante su implementación.

Objetivo del Back End CMS

El objetivo principal del Módulo 1 del proyecto *Predict Health* es desarrollar e implementar un sistema Back End CMS que permita gestionar de forma estructurada, segura y escalable los datos clínicos y operativos del sistema, dentro del plazo establecido del proyecto final. Este módulo se encarga de la lógica de negocio y del acceso a la base de datos, sobre la cual se construyen los demás componentes de la plataforma.

El CMS debe ofrecer operaciones CRUD sobre las principales entidades del sistema — pacientes, médicos, instituciones, historiales y predicciones —, garantizando la integridad de los datos a través de un modelo relacional normalizado hasta Tercera Forma Normal (3FN), conforme al diseño de base de datos definido. Asimismo, debe incorporar mecanismos de autenticación y control de acceso basados en roles, limitando las funcionalidades disponibles según el tipo de usuario (administrador, institución, médico o paciente).

Además, el módulo debe facilitar la generación de reportes dinámicos y la visualización de indicadores clave de desempeño (KPIs) clínicos mediante gráficas analíticas, con el fin de respaldar el análisis de tendencias, evaluación de resultados y la toma de decisiones tanto médicas como administrativas.

En conjunto, este módulo busca establecer una arquitectura sólida que asegure la integridad y seguridad de la información médica, habilite el crecimiento futuro del sistema y proporcione herramientas analíticas de valor para los distintos perfiles de usuario.

Funcionalidades implementadas

A continuación, se describen las principales características implementadas:

1. Gestión de Entidades

Se desarrollaron controladores y rutas protegidas para realizar operaciones CRUD (crear, leer, actualizar y eliminar) sobre las principales entidades del sistema. Cada entidad fue implementada con validaciones específicas y relaciones estructuradas con otras tablas del sistema.

Pacientes

- Registro de pacientes con información personal y datos clínicos iniciales.
- Edición y consulta del historial médico individual.
- Eliminación lógica (soft delete) manteniendo trazabilidad de los datos.
- Filtros de búsqueda por nombre, edad, institución o fecha de registro.

Doctores

- Alta de usuarios con rol médico, incluyendo especialidad, número de cédula profesional y datos de contacto.
- Asociación de pacientes al perfil de cada doctor.
- Edición de información profesional y consulta de actividad médica.
- Registro de diagnósticos realizados y revisión de predicciones automáticas.

Instituciones

- Registro de clínicas, hospitales u organizaciones de salud.
- Visualización del personal médico y pacientes asociados a cada institución.
- Acceso a reportes institucionales y gestión de recursos desde el sistema.

Usuarios

- Registro de usuarios con asignación de rol (administrador, doctor, institución o paciente).
- Autenticación segura y gestión de credenciales.
- Configuración y actualización de perfiles.
- Control de acceso basado en el tipo de usuario.

Diagnósticos y Predicciones

- Registro manual de diagnósticos médicos.
- Asociación de predicciones generadas por el sistema a pacientes específicos.
- Almacenamiento de métricas como nivel de confianza y resultado final.
- Relación directa con el historial clínico y el médico responsable.

2. Control de Acceso por Roles

La autenticación del sistema se implementó mediante **JSON Web Tokens (JWT)**. Se desarrollaron middlewares personalizados que validan cada solicitud según:

- Existencia y validez del token.
- Permisos asociados al rol del usuario autenticado.

Roles definidos y funciones principales:

Rol	Funciones permitidas
Administrador	Acceso completo: gestión de usuarios, instituciones, diagnósticos, predicciones, reportes y configuración general del sistema.
Doctor	Consulta y edición de pacientes asignados, creación de diagnósticos y revisión de predicciones clínicas.
Institución	Visualización de métricas propias, gestión de personal médico y acceso a reportes institucionales.
Paciente	Acceso al historial médico personal, métricas de salud y recomendaciones generadas por el sistema.

3. Reportes y Visualización de KPIs

Se incluyó una funcionalidad de generación de reportes basada en consultas dinámicas y segmentadas. Los endpoints correspondientes están disponibles únicamente para usuarios con rol administrador o institución, según el alcance de los datos.

KPIs desarrollados:

- Cantidad de pacientes registrados por institución.
- Enfermedades más frecuentes según el grupo de edad correspondiente.
- Proporción de diagnósticos positivos vs. negativos.
- Uso de la herramienta de predicción por doctor.
- Tiempo promedio entre diagnóstico clínico y predicción generada.
- Distribución de diagnósticos por especialidad médica y fechas.

Cada uno de estos indicadores cuenta con un endpoint específico que devuelve los datos procesados y listos para ser graficados desde el Front End, incluyendo filtros por fecha, tipo de usuario o institución.

4. Seguridad y Validación

Se incorporaron medidas de seguridad orientadas a proteger tanto los datos personales como la información clínica sensible. Entre las principales medidas implementadas se encuentran:

- Encriptación de contraseñas utilizando **bcrypt**.
- Validaciones estrictas en cada ruta del sistema (tipos de datos, campos requeridos, formato).
- Middleware global para gestión de errores, respuestas estándar y logs de actividad.

- Protección de endpoints sensibles según nivel de acceso y rol del usuario autenticado.

5. Lógica SQL: Procedimientos Almacenados y Vistas

Para encapsular lógica de negocio crítica, reducir la complejidad del backend y mejorar el rendimiento en operaciones de alto volumen, el sistema emplea intensivamente **Stored Procedures, queries SQL personalizadas y vistas SQL**.

Estos elementos son ejecutados desde los controladores del CMS para alimentar dashboards, validar datos, generar reportes y mantener la integridad de procesos sensibles como la creación de pacientes o el cálculo de métricas clínicas. A continuación se resumen sus propósitos generales:

Stored Procedures (Procedimientos almacenados)

- Agrupan operaciones relacionadas (por ejemplo, creación de pacientes con cuenta y perfil clínico).
- Ejecutan cálculos de KPIs directamente desde la base de datos (por médico, institución o sistema).
- Devuelven múltiples valores de salida (como totales, promedios, condiciones comunes, etc.).
- Aumentan la seguridad y eficiencia al centralizar lógica sensible en el servidor SQL.

Queries SQL personalizadas

- Se ejecutan desde el backend para obtener datos dinámicos filtrados.
- Usan parámetros en tiempo real (rango de fechas, regiones, especialidades).
- Proveen resultados optimizados para alimentar reportes y dashboards administrativos.
- Están organizadas por endpoint y cumplen con funciones específicas del sistema.

Vistas SQL (Views)

- Estructuran datos precalculados que alimentan gráficos e informes.
- Simplifican consultas complejas con múltiples joins y filtros.
- Mejoran el tiempo de respuesta del backend al reducir procesamiento en tiempo real.
- Se utilizan en módulos clave: validación de pacientes, distribución médica, registros mensuales, condiciones clínicas prevalentes y más.

6. Interfaz Administrativa de Pruebas

Durante el proceso de desarrollo, se creó una interfaz web de prueba con el objetivo de validar el comportamiento del sistema y facilitar las pruebas funcionales.

Esta interfaz permitió:

- Visualizar y editar entidades en tiempo real.
- Acceder a las gráficas basadas en los KPIs definidos.

- Simular el uso del CMS desde diferentes roles de usuario.

Aunque este avance no representa el Front End final del sistema, esta herramienta fue clave para demostrar la funcionalidad del módulo ante evaluadores y para validar que la lógica implementada respondía correctamente a distintos escenarios de uso.

Queries SQL y Stored Procedures

Empleamos **queries SQL**, **procedimientos almacenados** y **vistas SQL** para encapsular la lógica de negocio, mejorar la eficiencia de las operaciones y soportar los módulos de reportes y visualización de KPIs. A continuación se describen estos con fragmentos de código en la mayoría de estos:

1. Stored Procedures (Procedimientos Almacenados)

a) sp_create_patient_with_profile

Crea un paciente junto con su cuenta de usuario y perfil de salud en una única transacción.

SQL

```
CREATE OR REPLACE PROCEDURE sp_create_patient_with_profile(
    p_first_name VARCHAR(100),
    p_last_name  VARCHAR(100),
    p_email      VARCHAR(255),
    p_password_hash VARCHAR(255),
    p_date_of_birth DATE,
    p_gender     VARCHAR(20),
    p_phone      VARCHAR(20),
    p_doctor_id  UUID,
    p_institution_id UUID,
    p_height_cm  DECIMAL(5,2),
    p_weight_kg  DECIMAL(5,2),
    p_blood_type VARCHAR(5)
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_patient_id UUID;
    v_user_id     UUID;
BEGIN
    INSERT INTO patients (...) RETURNING id INTO v_patient_id;
    INSERT INTO users (...) RETURNING id INTO v_user_id;
    INSERT INTO health_profiles (...);
    COMMIT;
END;
$$;
```

Uso: Simplifica el registro completo de pacientes con relaciones entre tablas.

Ventaja: Reduce errores y mantiene la integridad referencial.

b) sp_get_patient_stats_by_month

Genera estadísticas clínicas básicas por mes y año.

```
SQL
CREATE OR REPLACE PROCEDURE sp_get_patient_stats_by_month(
    p_year INTEGER,
    OUT total_patients INTEGER,
    OUT new_patients INTEGER,
    OUT validated_patients INTEGER,
    OUT avg_age DECIMAL(5,2)
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_year INTEGER := COALESCE(p_year, EXTRACT(YEAR FROM
CURRENT_DATE));
BEGIN
    SELECT COUNT(*) INTO total_patients FROM patients WHERE is_active
= TRUE;
    SELECT COUNT(*) INTO new_patients FROM patients WHERE ...;
    SELECT COUNT(*) INTO validated_patients FROM patients WHERE ...;
    SELECT AVG(EXTRACT(YEAR FROM AGE(...))) INTO avg_age FROM
patients WHERE ...;
END;
$$;
```

Uso: Dashboard de crecimiento y validación.

Ventaja: Agrupa métricas clave en una sola llamada.

c) sp_get_doctor_performance_stats

Muestra el rendimiento de un médico (o todos si se omite el ID).

```
SQL
CREATE OR REPLACE PROCEDURE sp_get_doctor_performance_stats(
    p_doctor_id UUID,
    OUT total_patients INTEGER,
    OUT avg_patient_age DECIMAL(5,2),
    OUT common_conditions TEXT
)
LANGUAGE plpgsql
AS $$
BEGIN
    SELECT COUNT(*) INTO total_patients FROM patients WHERE ...;
    SELECT AVG(...) INTO avg_patient_age FROM patients WHERE ...;
```

```

        SELECT STRING_AGG(...) INTO common_conditions FROM patients JOIN
health_profiles ...;
END;
$$;

```

Uso: KPI individual o global de desempeño médico.

Ventaja: Retorna múltiples métricas relevantes en un solo llamado.

d) `sp_get_institution_analytics`

Métricas agrupadas por institución médica.

```

SQL
CREATE OR REPLACE PROCEDURE sp_get_institution_analytics(
    p_institution_id UUID,
    OUT patient_count INTEGER,
    OUT doctor_count INTEGER,
    OUT avg_consultation_fee DECIMAL(10,2),
    OUT most_common_specialty VARCHAR(100)
)
LANGUAGE plpgsql
AS $$
BEGIN
    SELECT COUNT(*) INTO patient_count FROM patients WHERE ...;
    SELECT COUNT(*) INTO doctor_count FROM doctors WHERE ...;
    SELECT AVG(consultation_fee) INTO avg_consultation_fee FROM
doctors WHERE ...;
    SELECT ds.name INTO most_common_specialty FROM doctors JOIN
doctor_specialties ... LIMIT 1;
END;
$$;

```

Uso: Reportes de rendimiento institucional.

Ventaja: Resumen compacto con datos clínicos, financieros y de recursos humanos.

2. Vistas SQL para Reportes y Dashboards

a) `vw_patient_demographics`

```

SQL
CREATE OR REPLACE VIEW vw_patient_demographics AS

```

```

SELECT
    p.id, p.first_name, p.last_name, p.email,
    EXTRACT(YEAR FROM AGE(CURRENT_DATE, p.date_of_birth)) AS age,
    p.gender, p.validation_status, mi.name AS institution_name,
    hp.blood_type, hp.hypertension_diagnosis, ...
FROM patients p
LEFT JOIN doctors d ON p.doctor_id = d.id
LEFT JOIN medical_institutions mi ON p.institution_id = mi.id
LEFT JOIN health_profiles hp ON p.id = hp.patient_id
WHERE p.is_active = TRUE;

```

Uso: Datos para gráficas demográficas.

Contiene: Edad, género, diagnóstico y afiliación institucional.

b) vw_doctor_performance

```

SQL
CREATE OR REPLACE VIEW vw_doctor_performance AS
SELECT
    d.id, d.first_name, d.last_name, ds.name AS specialty,
    COUNT(p.id) AS patient_count,
    AVG(EXTRACT(YEAR FROM AGE(CURRENT_DATE, p.date_of_birth))) AS
avg_patient_age
FROM doctors d
LEFT JOIN patients p ON d.id = p.doctor_id
...
GROUP BY d.id, ds.name;

```

Uso: Análisis comparativo entre médicos.

KPI: Número de pacientes, edad promedio, especialidad, institución.

c) vw_monthly_registrations

```

SQL
CREATE OR REPLACE VIEW vw_monthly_registrations AS
SELECT
    DATE_TRUNC('month', created_at) AS registration_month,
    COUNT(*) AS total_registrations,
    COUNT(CASE WHEN validation_status = 'full_access' THEN 1 END) AS
validated_registrations
FROM patients

```

```
WHERE is_active = TRUE
GROUP BY registration_month
ORDER BY registration_month DESC;
```

Uso: Tendencias de ingreso y validación.

Ideal para: Gráficas de líneas y comparativas.

d) vw_health_condition_stats, vw_dashboard_overview, vw_doctor_specialty_distribution, vw_geographic_distribution, etc.

Cada una de estas vistas concentra datos para generar KPIs específicos:

- **Condiciones de salud prevalentes** (vw_health_condition_stats)
- **Totales del sistema** (vw_dashboard_overview)
- **Distribución de especialidades médicas** (vw_doctor_specialty_distribution)
- **Distribución geográfica por estado** (vw_geographic_distribution)
- **Estado de validación de pacientes** (vw_patient_validation_status)

Estas vistas permiten que el frontend consuma datos preprocesados para construir gráficas complejas con bajo impacto en la base de datos. Para poder ver el código de las mismas, checar el archivo **init.sql** en el repositorio de GitHub.

3. Queries SQL

Q1: Query de Dashboard - Resumen General

Ubicación: [cms-backend/app/routes/dashboard.py](#) (línea 13)

```
SQL
SELECT * FROM vw_dashboard_overview;
```

Propósito: Obtener métricas principales para el dashboard administrativo: totales de pacientes, doctores, instituciones, usuarios y otros KPIs.

Q2: Query de Registros Mensuales

Ubicación: [cms-backend/app/routes/dashboard.py](#) (líneas 24–31)

```
SQL
SELECT
    TO_CHAR(registration_month, 'Mon YYYY') AS month,
```

```
total_registrations AS count  
  
FROM vw_monthly_registrations  
  
ORDER BY registration_month DESC  
  
LIMIT 6;
```

Propósito: Proveer datos para un gráfico de barras que muestra registros mensuales recientes.

Q3: Query de Distribución por Especialidades

Ubicación: [cms-backend/app/routes/dashboard.py](#) (líneas 42–49)

```
SQL  
SELECT  
  
    specialty,  
  
    doctor_count  
  
FROM vw_doctor_specialty_distribution  
  
WHERE doctor_count > 0  
  
ORDER BY doctor_count DESC  
  
LIMIT 8;
```

Propósito: Gráfico de especialidades médicas con mayor número de doctores activos.

Q4: Query de Distribución Geográfica

Ubicación: [cms-backend/app/routes/dashboard.py](#) (líneas 52–59)

```
SQL  
SELECT  
  
    region_state,  
  
    institution_count,  
  
    doctor_count,  
  
    patient_count  
  
FROM vw_geographic_distribution
```

```
ORDER BY institution_count DESC;
```

Propósito: Crear visualizaciones tipo mapa de calor o tablas por región.

Q5: Query de Condiciones de Salud

Ubicación: [cms-backend/app/routes/dashboard.py](#) (líneas 61–67)

```
SQL
SELECT
    condition,
    patient_count
FROM vw_health_condition_prevalence
ORDER BY patient_count DESC;
```

Propósito: Visualizar condiciones de salud más comunes (hipertensión, diabetes, colesterol alto).

Q6: Query de Estado de Validación

Ubicación: [cms-backend/app/routes/dashboard.py](#) (líneas 69–76)

```
SQL
SELECT
    validation_status,
    patient_count,
    percentage
FROM vw_patient_validation_status
ORDER BY patient_count DESC;
```

Propósito: Analizar el estado de validación de pacientes (porcentaje y conteo).

Q7: Query de Reporte de Pacientes

Ubicación: [cms-backend/app/routes/reports.py](#) (líneas 50–97)

SQL

SELECT

id, first_name, last_name, email, age, gender, validation_status,
doctor_first_name, doctor_last_name, institution_name,
blood_type,

hypertension_diagnosis, diabetes_diagnosis,
high_cholesterol_diagnosis, created_at

FROM vw_patient_demographics

WHERE 1=1

AND created_at >= :date_from

AND created_at <= :date_to

AND validation_status = :validation_status

AND gender = :gender

AND institution_name ILIKE :region

ORDER BY created_at DESC

LIMIT :limit OFFSET :offset;

Propósito: Generar reportes detallados y filtrables de pacientes con base en múltiples criterios.

Q8: Query de Reporte de Doctores

Ubicación: [cms-backend/app/routes/reports.py](#) (líneas 148–195)

SQL

SELECT

id, first_name, last_name, email, medical_license,
years_experience,

consultation_fee, specialty, institution_name, patient_count,
avg_patient_age, created_at

FROM vw_doctor_performance

WHERE 1=1

AND created_at >= :date_from

AND created_at <= :date_to


```
AND specialty ILIKE :specialty  
AND professional_status = :status  
AND institution_name ILIKE :region  
ORDER BY created_at DESC  
LIMIT :limit OFFSET :offset;
```

Propósito: Reporte dinámico de doctores, incluyendo métricas clave, filtros por región, especialidad y estado profesional.

KPIs, métricas y gráficas

A continuación, se describen los principales KPIs implementados, las métricas que analizan, su propósito, tipo de visualización y una breve interpretación basada en datos de ejemplo registrados en el sistema:

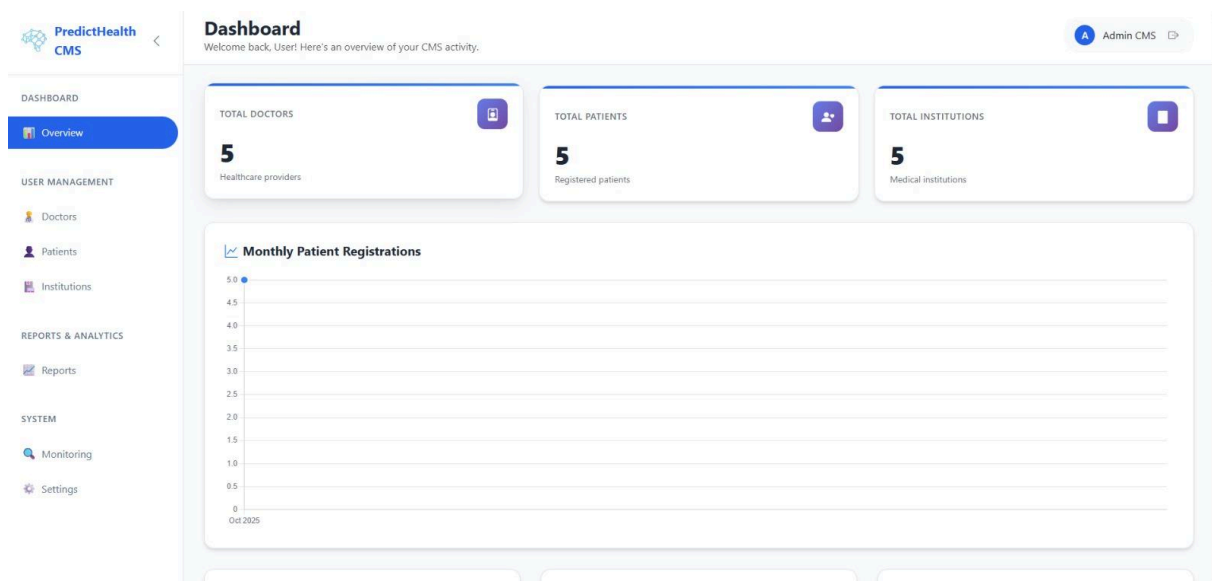
KPI 1: Total de registros por mes

- **Descripción:** Muestra la cantidad de nuevos pacientes registrados en cada mes.
- **Tipo de visualización:** Gráfico de líneas.

Interpretación:

La gráfica correspondiente a este KPI presenta una línea temporal que refleja el número de pacientes registrados mes a mes. Actualmente, se muestra un solo punto de actividad en octubre de 2025 con un total de cinco registros. Si bien el volumen es aún limitado, se confirma que el sistema está capturando correctamente los datos en tiempo real. En el futuro, esta métrica permitirá identificar tendencias de crecimiento, estacionalidades y evaluar el impacto de campañas o acciones institucionales.

●

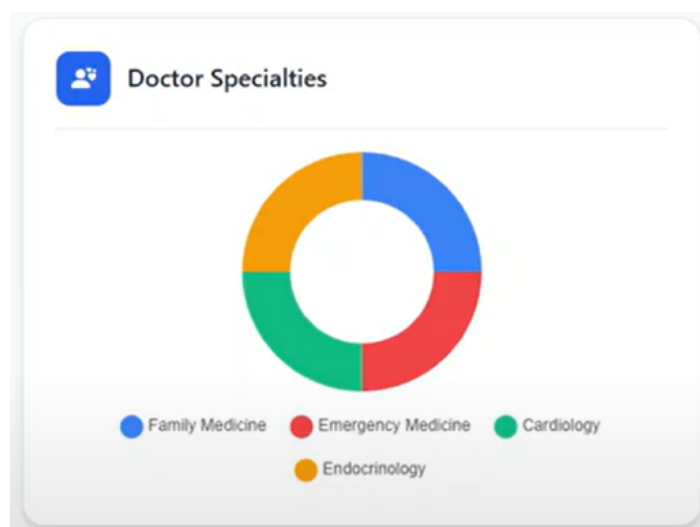


KPI 2: Distribución de especialidades médicas

- **Descripción:** Representa la cantidad de doctores activos por especialidad médica.
- **Tipo de visualización:** Gráfico de dona (interactivo con valores al pasar el cursor).

Interpretación:

La gráfica de dona muestra una distribución balanceada entre cuatro especialidades: *Family Medicine*, *Emergency Medicine*, *Cardiology* y *Endocrinology*. Aunque los porcentajes no son visibles directamente, se revelan al interactuar con el gráfico. Esta visualización permite evaluar la diversidad en la atención médica y detectar áreas con mayor o menor representación, facilitando decisiones de contratación y planificación médica.



KPI 3: Distribución geográfica de instituciones

- **Descripción:** Muestra el número de instituciones registradas por estado o región, incluyendo también el conteo de doctores y pacientes por ubicación.
- **Tipo de visualización:** Tarjetas estadísticas.

Interpretación:

Actualmente, Jalisco cuenta con dos instituciones registradas, mientras que Ciudad de México, Guanajuato y Nuevo León tienen una cada una. Aunque no se utiliza una gráfica convencional, el diseño en tarjetas permite una lectura clara y rápida de la distribución territorial. Esta información es clave para evaluar la cobertura del sistema, identificar zonas con mayor actividad e implementar estrategias de expansión o equilibrio regional.



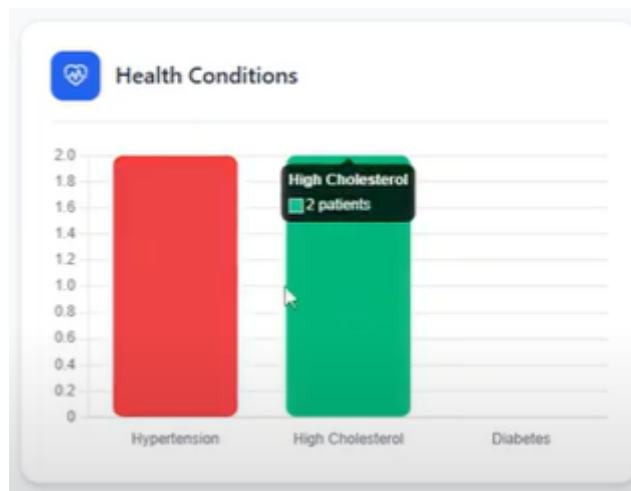
KPI 4: Condiciones de salud

- **Descripción:** Muestra el número de pacientes diagnosticados con hipertensión, diabetes o colesterol alto.
- **Tipo de visualización:** Gráfico de barras.

Interpretación:

El gráfico presenta la prevalencia de condiciones como hipertensión y colesterol alto, ambas con dos casos registrados, mientras que diabetes aún no tiene pacientes diagnosticados. La

visualización incluye interactividad que muestra el conteo exacto al posicionarse sobre cada barra. Este KPI es útil para identificar tendencias clínicas dentro del sistema y priorizar acciones preventivas o ajustes en la atención médica.

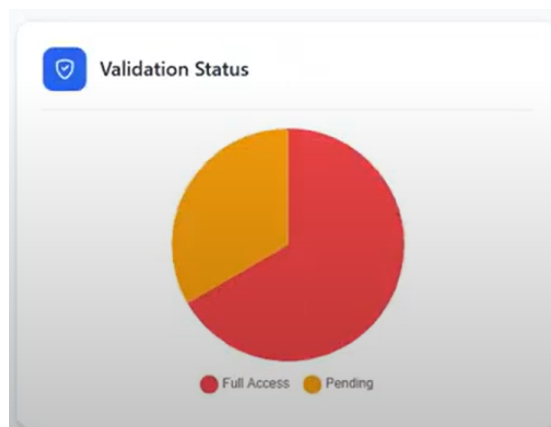


KPI 5: Estado de validación de pacientes

- **Descripción:** Muestra el porcentaje y conteo de pacientes por estado de validación (Full Access, Pending, etc.).
- **Tipo de visualización:** Gráfico de pastel.

Interpretación:

La gráfica de pastel indica que una parte importante de los pacientes ya completó su validación médica, mientras que otro segmento aún está pendiente. Esta métrica es fundamental, ya que sólo los pacientes con validación completa pueden acceder a funcionalidades críticas del sistema, como recomendaciones clínicas automatizadas mediante IA. El KPI permite monitorear el avance del onboarding clínico y la eficiencia institucional en la verificación de usuarios.

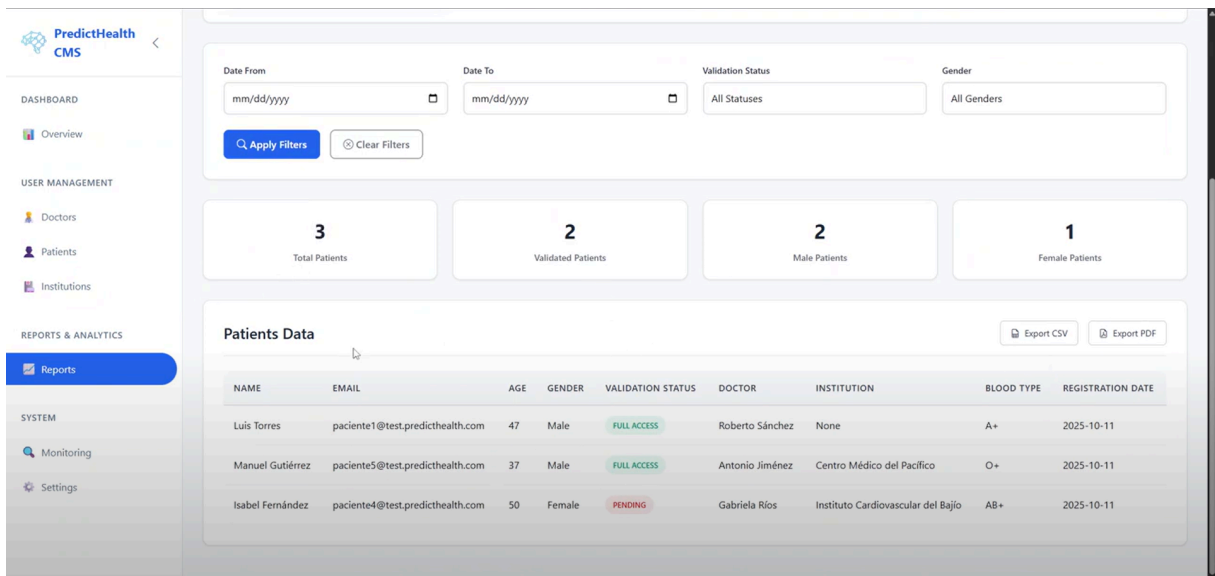
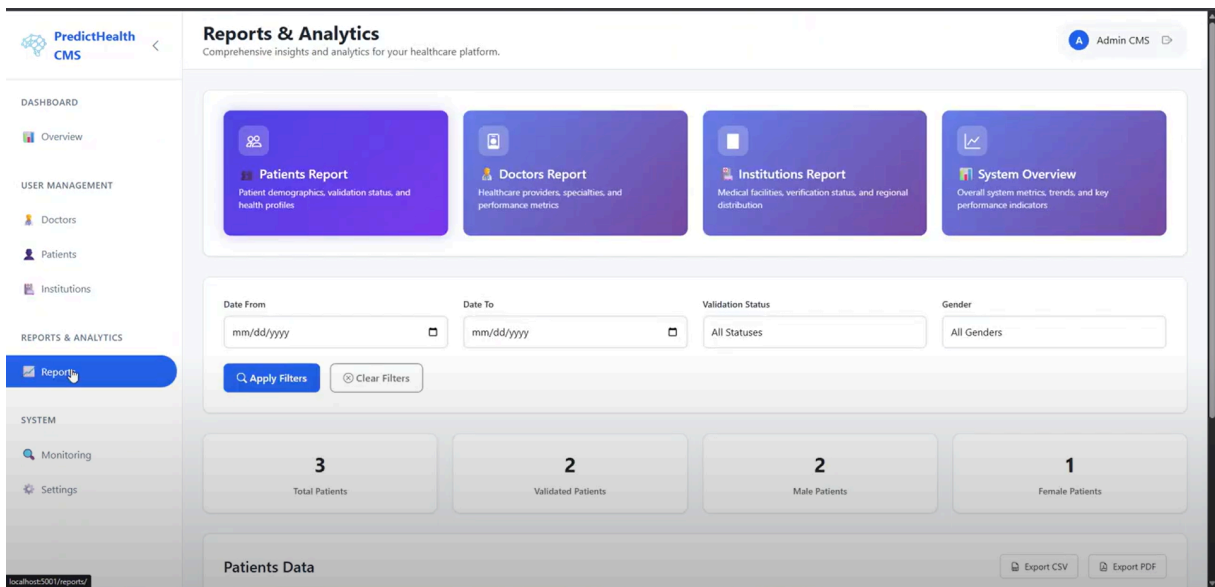


KPI 6: Reportes y analíticas detalladas por tipo de usuario

- **Descripción:** Genera tablas y visualizaciones con filtros por fecha, género, institución, especialidad, estado de validación y más.
- **Tipo de visualización:** Tabla dinámica con filtros + gráficas auxiliares según campo.

Interpretación:

Este módulo permite a los usuarios explorar datos de pacientes, doctores e instituciones mediante filtros personalizables. Cada reporte incluye tarjetas resumen (por ejemplo, total de pacientes validados, género, etc.) y una tabla detallada que se puede exportar a CSV o PDF. Esta funcionalidad es clave para tareas administrativas, auditorías, análisis poblacional y toma de decisiones clínicas o estratégicas. Al ser transversal, aplica a múltiples tipos de usuario en el sistema.



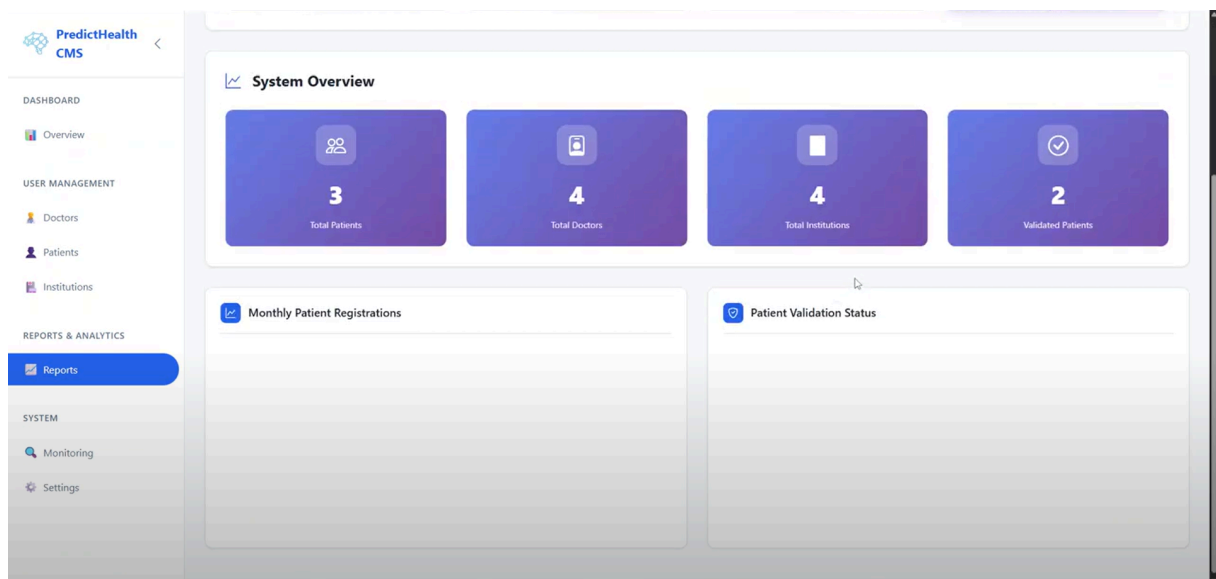
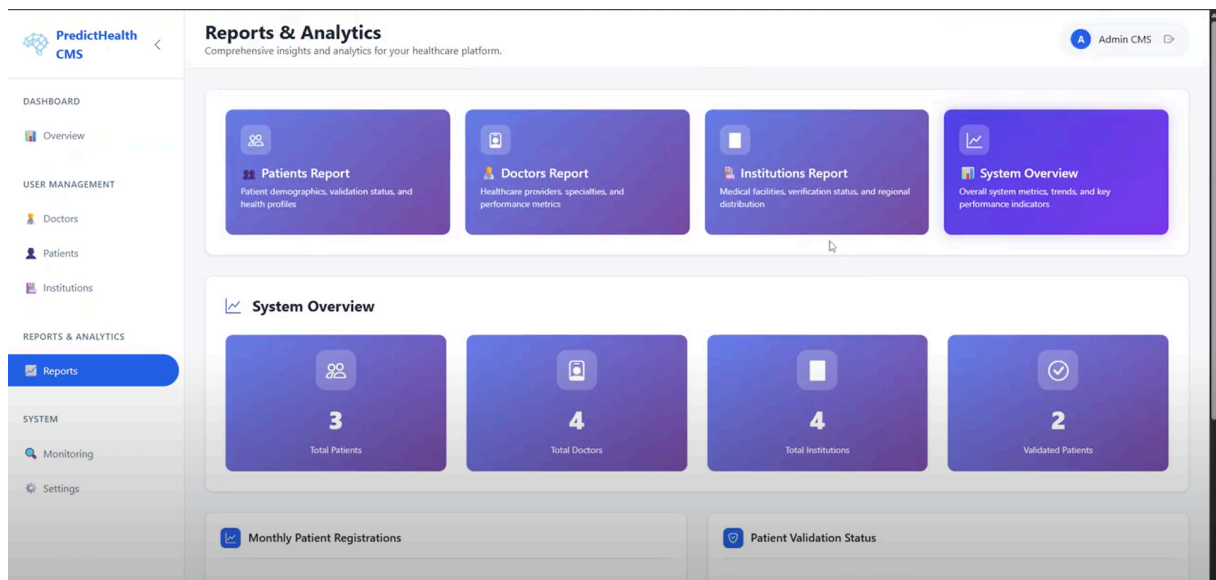
KPI 6.1: Vista general del sistema (System Overview)

- **Descripción:** Conjunto de tarjetas con métricas clave del sistema (total de pacientes, doctores, instituciones y pacientes validados).
- **Tipo de visualización:** Tarjetas estáticas de resumen + gráficas auxiliares.

Interpretación:

La vista System Overview consolida indicadores esenciales del sistema en una sola pantalla, facilitando una lectura inmediata del estado actual de la plataforma Predict Health. Esto incluye métricas de volumen y avance en procesos clave como la validación de pacientes.

La utilidad de este KPI radica en ofrecer una panorámica de alto nivel a los administradores, sin necesidad de aplicar filtros o navegar por reportes individuales. Complementa otros KPIs al presentar un resumen operativo en tiempo real.



Prototipo en funcionamiento

El Back End CMS se encuentra completamente funcional y desplegado en un entorno de desarrollo local, permitiendo la administración integral del sistema mediante una interfaz robusta, organizada por roles y conectada directamente a la base de datos mediante procedimientos almacenados y vistas optimizadas.

A continuación se describen las principales áreas funcionales del prototipo y su estado actual:

1. Autenticación y Control de Acceso por Roles

El sistema implementa autenticación basada en JWT y control de acceso según tipo de usuario (Administrador, Institución, Doctor, Paciente). Cada rol tiene acceso restringido a funciones específicas. Por ejemplo:

- **Administradores:** Control total del sistema, configuración de usuarios, instituciones y acceso a todos los reportes.
- **Instituciones:** Administración de su personal médico, visualización de KPIs institucionales.
- **Doctores:** Consulta y gestión de pacientes asignados, perfiles clínicos y diagnósticos.
- **Pacientes:** Acceso a su propio historial, estado de validación y recomendaciones médicas (solo si tienen validación completa).

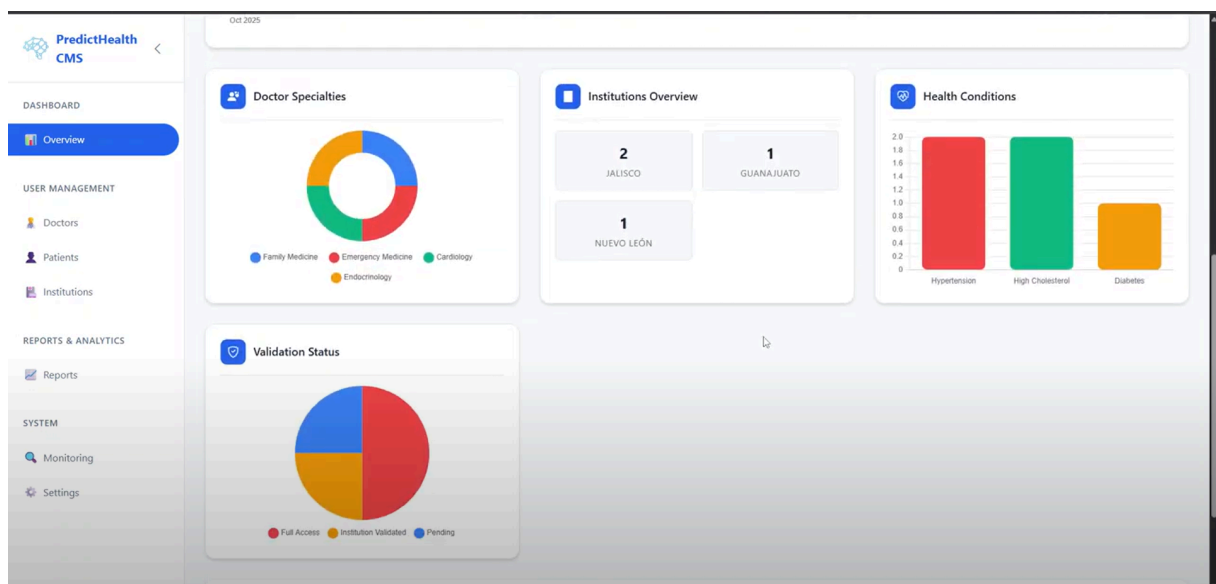


Imagen 1. Dashboard del administrador

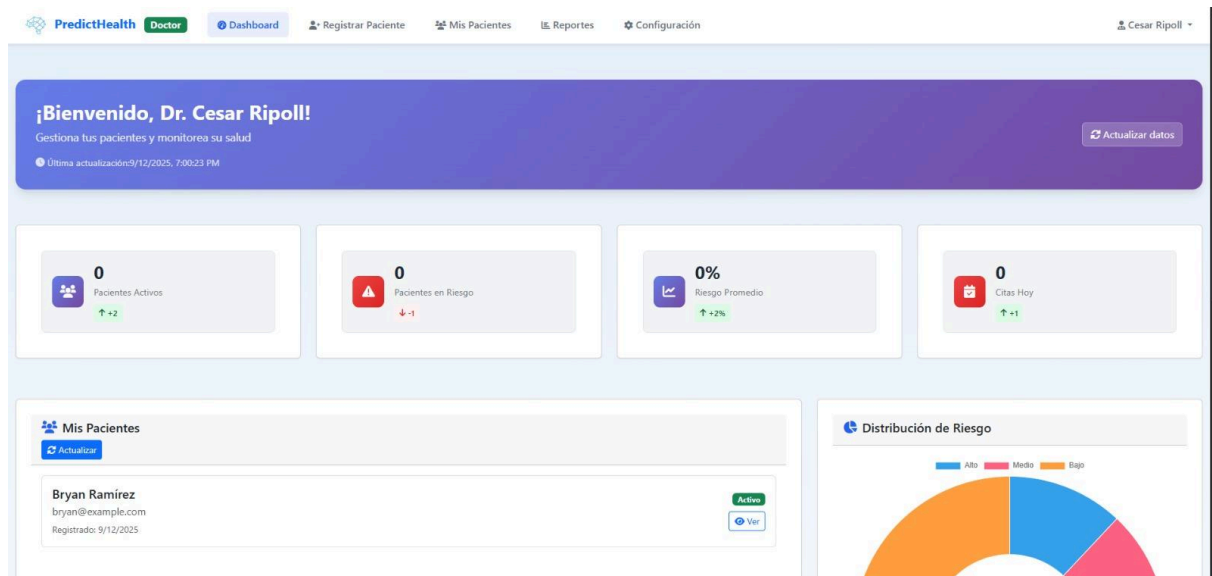


Imagen 2. Dashboard del doctor

2. Interfaz de Gestión de Usuarios

El CMS cuenta con secciones bien diferenciadas para:

- **Doctores:** Registro, edición, y visualización de información profesional y desempeño.
- **Pacientes:** Datos demográficos, estado de salud, diagnósticos y estatus de validación.
- **Instituciones:** Registro de clínicas/hospitales, personal médico y asignaciones regionales.

Name	Email	Date of Birth	Gender	Doctor	Institution	Validation Status	Actions
Javier Ruiz paciente3@test.predicthealth.com	paciente3@test.predicthealth.com	Nov 08, 1990	Male	Not assigned	Not assigned	Institution Validated	[Edit] [Delete]
Isabel Fernández paciente4@test.predicthealth.com	paciente4@test.predicthealth.com	May 30, 1975	Female	Not assigned	Not assigned	Pending	[Edit] [Delete]
Manuel Gutiérrez paciente5@test.predicthealth.com	paciente5@test.predicthealth.com	Sep 14, 1988	Male	Not assigned	Not assigned	Full Access	[Edit] [Delete]
Luis Torres paciente1@test.predicthealth.com	paciente1@test.predicthealth.com	Mar 12, 1978	Male	Not assigned	Not assigned	Full Access	[Edit] [Delete]

Imagen 3. Sección de pacientes desde el rol de administrador

3. Módulo de Reportes y Analíticas

Incluye un sistema avanzado de filtros y exportación de datos con visualizaciones dinámicas. Los reportes se dividen por tipo:

- **Reporte de Pacientes**

- **Reporte de Doctores**
- **Reporte de Instituciones**
- **Vista General del Sistema**

Cada uno incluye tarjetas resumen, tablas exportables y KPIs personalizados, todo alimentado desde vistas SQL complejas diseñadas para optimizar el rendimiento.

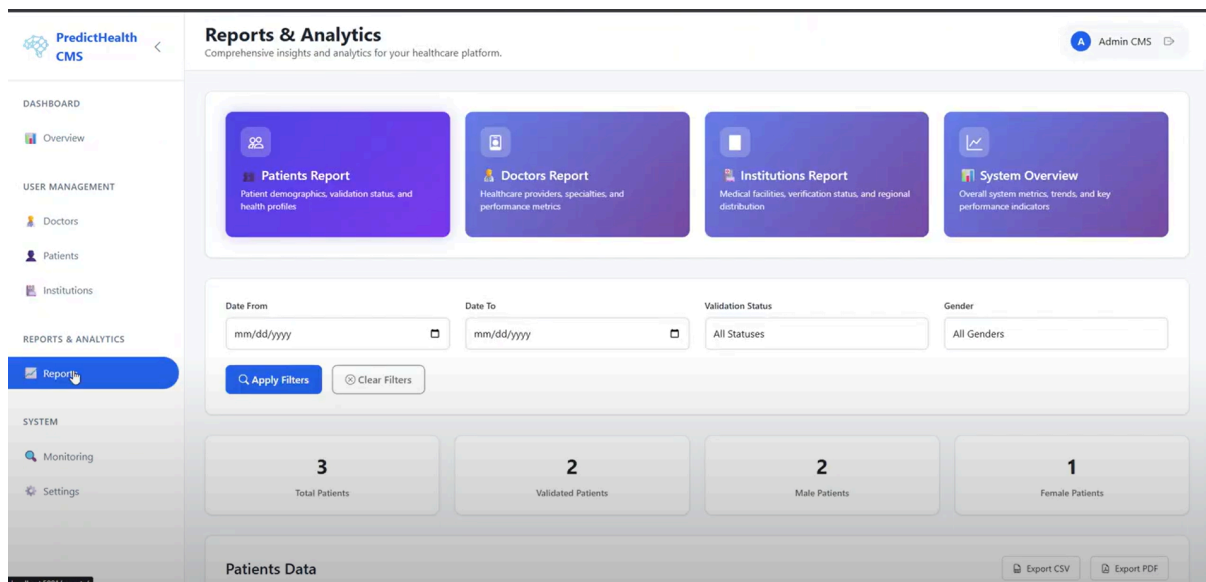


Imagen 4. Tarjetas para seleccionar el tipo de reporte

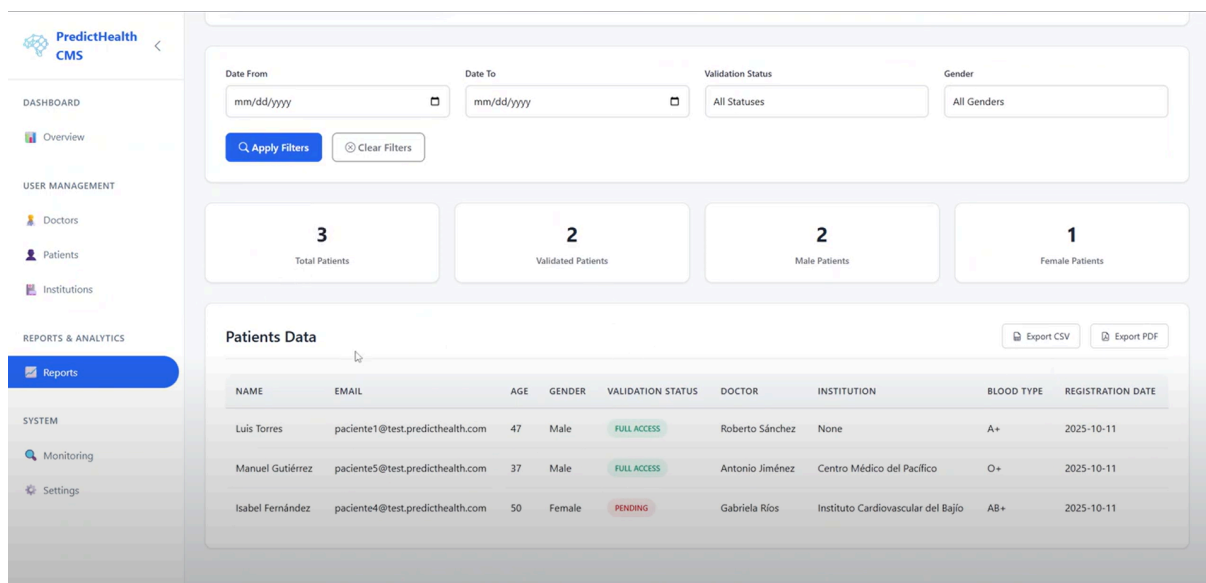


Imagen 5. Reporte general de todos los pacientes

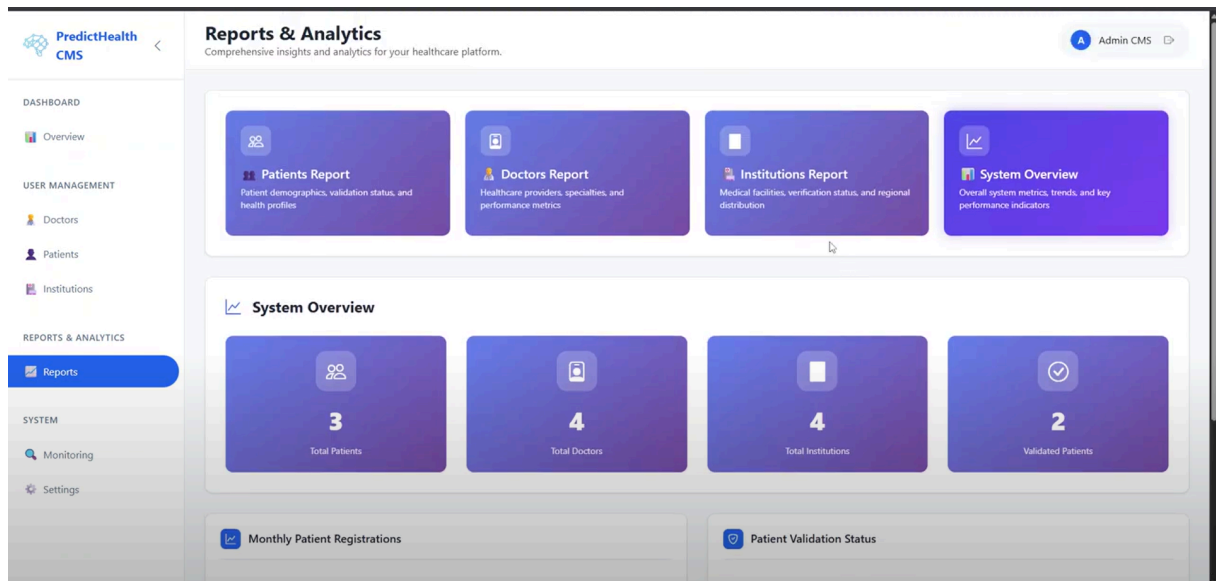


Imagen 6. Vista general del sistema

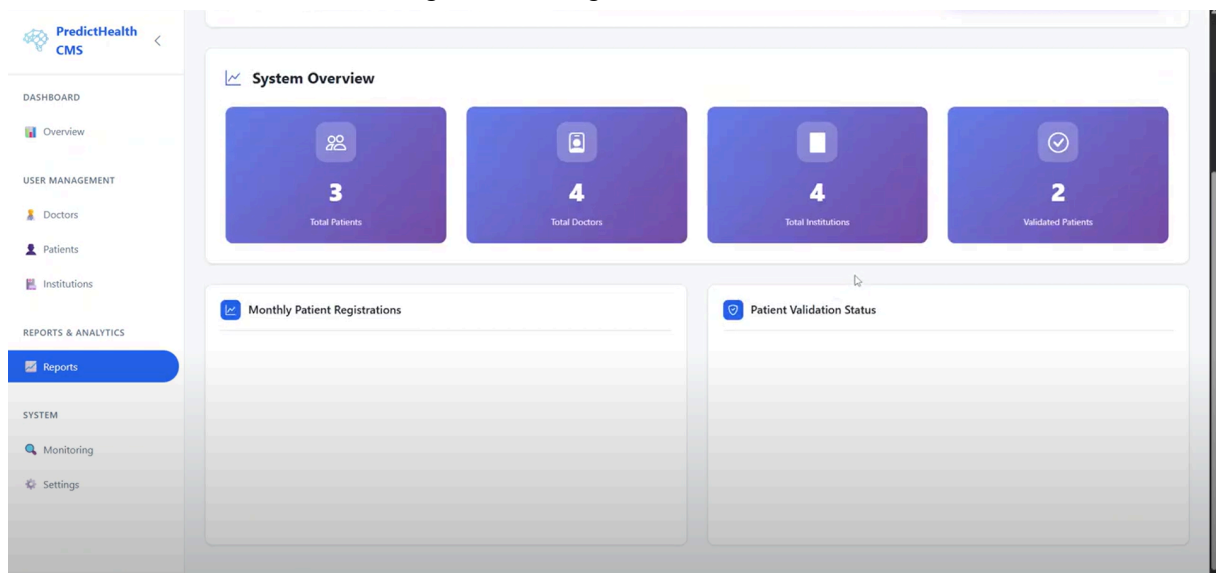


Imagen 7. Gráficas del registro y estatus de validación de los pacientes en el sistema

4. Visualización de KPIs Clínicos y Operativos

Se integran al menos seis gráficas interactivas basadas en KPIs críticos del sistema (ver secciones anteriores), visualizadas desde el dashboard principal y el módulo de reportes. Incluyen:

- Registros mensuales
- Especialidades médicas
- Validación de pacientes
- Condiciones de salud prevalentes
- Distribución geográfica de instituciones
- Métricas del sistema general

5. Monitoreo en Tiempo Real del Sistema

El CMS cuenta con una vista específica para **monitorización del estado de los microservicios**, lo que permite verificar en tiempo real si cada módulo del sistema (auth,

backend, base de datos, Redis, etc.). Esta herramienta es clave para la administración técnica del mismo, por lo que, esta sección muestra información del estado actual de cada microservicio, incluyendo nombre, puerto, host, tiempo de respuesta y si se encuentra activo en formato de tarjetas.

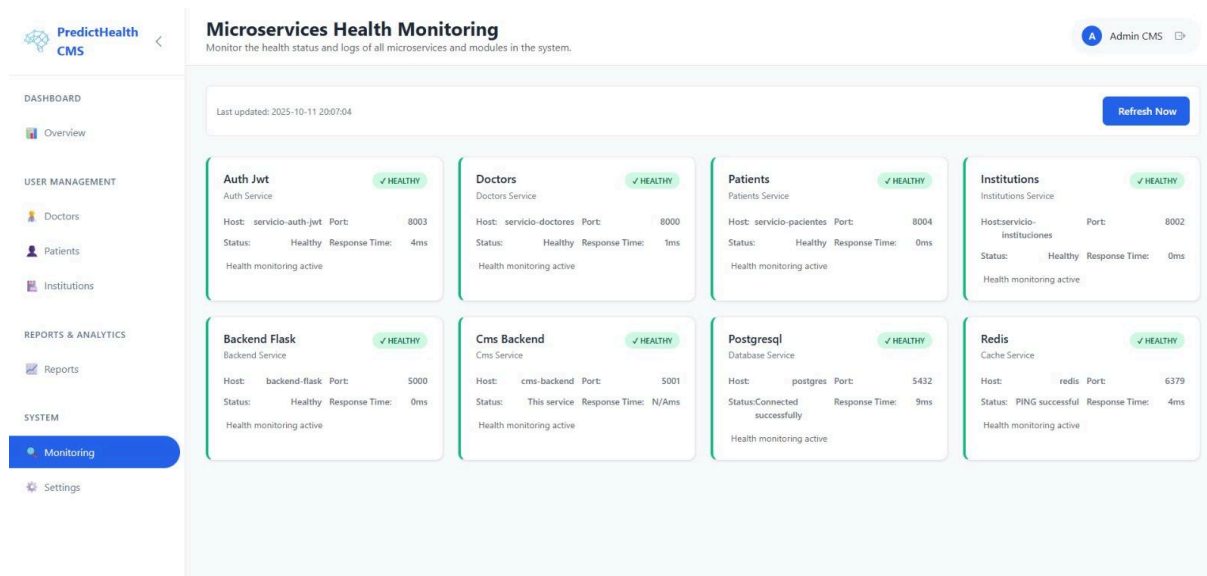


Imagen 8. Monitoreo de los microservicios y módulos del proyecto

6. Exportación de Reportes

Todos los reportes del sistema pueden ser exportados en formato **CSV** o **PDF**, lo que facilita su uso para propósitos administrativos, clínicos o de auditoría y se encuentran en la parte superior derecha de la tabla interactiva de cada reporte.

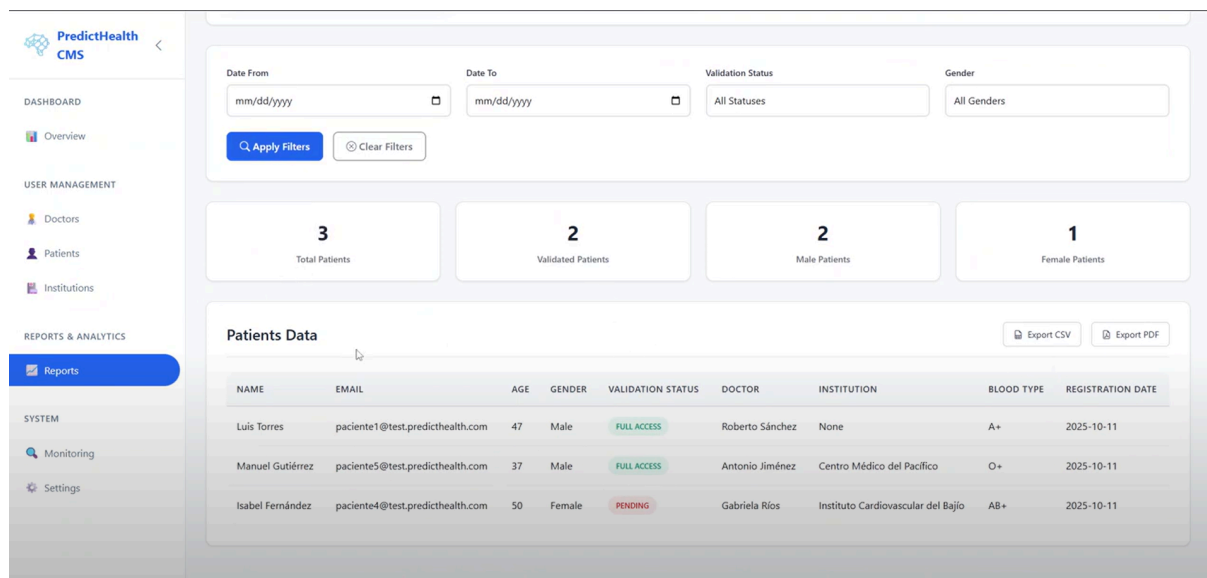


Imagen 9. Exportación de reportes en formato PDF o CSV

7. Edición de Datos de Usuarios

El CMS permite la edición completa de los datos de pacientes, doctores e instituciones a través de formularios estructurados y validados. En la vista de edición, se pueden modificar campos personales (nombre, fecha de nacimiento, género, contacto), así como la asociación

médica (doctor primario e institución asignada). Esta funcionalidad es esencial para mantener actualizada la información clínica y operativa del sistema, facilitando una gestión precisa por parte de los administradores o médicos responsables.

PredictHealth CMS

Edit Patient

Update information for Javier Ruiz

Personal Information

First Name *
Javier

Last Name *
Ruiz

Email Address *
paciente3@test.predicthealth.com

Phone Number *
52-33-4003-0001

Date of Birth *
11/08/1990

Gender
Male

Medical Association

Primary Doctor
Fernando Vázquez

Medical Institution
Centro de Salud Preventiva Sur

Imagen 10. Formulario para editar los datos de un paciente desde el administrador.

Para visualizar el funcionamiento completo del prototipo, incluyendo flujos de navegación, operaciones CRUD, visualización de gráficas y monitoreo del sistema, se ha preparado el siguiente video demostrativo: <https://www.youtube.com/watch?v=NdJMmJfloHs>

Conclusiones

El desarrollo del módulo Back End CMS del proyecto *Predict Health* representó una experiencia integral en la construcción de una solución tecnológica orientada al manejo estructurado de datos clínicos. A lo largo del proceso se logró implementar un sistema robusto que no solo cumple con los requerimientos funcionales previstos —como la gestión de usuarios, autenticación por roles, y operaciones CRUD sobre las entidades clave—, sino que también incorpora capacidades avanzadas como la generación de reportes dinámicos, visualización de KPIs, y consultas optimizadas mediante procedimientos almacenados y vistas SQL.

Uno de los principales logros fue la integración de una arquitectura modular basada en principios de separación de responsabilidades, lo que permitió escalar las funcionalidades sin comprometer la estabilidad del sistema. Esta arquitectura, además de facilitar el mantenimiento, sienta las bases para futuras extensiones como integración con IA o sistemas externos con acoplamientos débiles.

Durante el desarrollo nos enfrentamos a diversos desafíos técnicos. La gestión de la lógica compleja dentro de la base de datos, a través de stored procedures, exigió una planificación cuidadosa para evitar duplicidad de operaciones, asegurar la consistencia entre tablas relacionadas, y reducir al mínimo los tiempos de respuesta en consultas de gran volumen. La implementación del control de acceso por roles también presentó un reto importante, ya que cada tipo de usuario debía tener visibilidad y permisos específicos, sin comprometer la seguridad ni la experiencia de uso. Este componente se resolvió combinando validaciones JWT en los endpoints con filtros lógicos en las consultas SQL.

Otro aspecto crítico fue la sincronización entre los datos generados por la capa de persistencia y su presentación visual en dashboards. Para lograrlo, se diseñaron vistas especializadas que entregan datos listos para graficarse, reduciendo la lógica del lado del cliente y garantizando que las visualizaciones fueran coherentes con los filtros aplicados (fecha, usuario, institución, etc.). Esta decisión de diseño no solo mejoró el rendimiento, sino que también simplificó el desarrollo del front end.

Entre las principales lecciones aprendidas se destaca la necesidad de estructurar desde el inicio una base de datos relacional bien normalizada, acompañada de documentación clara y estructuras reutilizables. Esto resultó fundamental para garantizar la integridad de los datos y facilitar el trabajo colaborativo entre módulos. Además, quedó demostrado que la utilización de stored procedures y vistas, aunque demanda una inversión inicial en diseño, aporta grandes beneficios a nivel de eficiencia y mantenimiento.

Finalmente, el uso de una interfaz administrativa de prueba permitió validar todas las funcionalidades antes del despliegue final. Esta herramienta fue clave no solo para detectar errores en etapas tempranas, sino también para demostrar la operación del sistema ante terceros y simular casos de uso reales bajo distintos perfiles de usuario.

En conclusión, este módulo no solo representa una implementación exitosa, sino que constituye una base técnica sólida sobre la cual podrá evolucionar un sistema clínico integral, extensible y orientado a la toma de decisiones basada en datos.